



南开大学
Nankai University

南 开 大 学

计 算 机 学 院
并行程序设计期末实验报告

PCFG 口令猜测并行化综合实验

刘华彬

年级：大二

专业：计算机科学与技术

指导教师：崔立骁

2026 年 7 月 23 日

摘要

本文以 PCFG 口令猜测程序为对象，围绕 Train、Guess 和 Hash 三个阶段，整合了本学期 SIMD、多线程、MPI 和 GPU 并行编程实验的实现与结果。前序实验中，Hash 阶段利用不同候选口令之间的独立性，将 MD5 改写为 4 路 ARM NEON 向量计算，并通过 AoS 到 SoA 的数据布局转换提高向量装载效率；在 -O2 下，10M 候选的 Hash time 由 2.95s 降至 1.05s。Guess 阶段采用任务规模阈值、预分配数组直接写入和优先队列堆化，将仅使用 SIMD 哈希时的 Guess time 由 0.616s 降至 0.198s。Train 阶段利用频率统计的可加性进行局部模型训练和合并，并结合 MPI 训练分片、主进程-工作进程分工以及生成-哈希流水线；在 MPI_NP=4,OMP_NUM_THREADS=8 的前序融合版本中，完整程序平均总时间为 8.631917s，相比串行基线的 30.538141s 获得约 3.54 倍加速。

本次期末实验进一步针对 MPI Hash 分发和 GPU 候选生成路径进行优化。针对原 MPI 方案中完整候选字符串反复打包、传输和恢复 `vector<string>` 的问题，改为发送 PT 模板和值范围，由工作进程在本地生成对应候选并完成哈希，减少了通信开销。针对前序 CUDA 实验中仅生成候选字符串、仍需回传字符串并在 CPU 上计算 MD5 的问题，本次将短候选的生成和单块 MD5 融合到同一个 CUDA kernel，Host 只接收固定长度摘要。该方案显著减少了 CPU 端 MD5 和明文字符串回传，但端到端性能仍受 Host 侧打包、数据传输、任务提交和 PT 队列维护影响。仓库地址: https://github.com/BLH549/NKU_Parallel

目录

一、 问题描述与整合目标	1
(一) PCFG 口令猜测程序结构	1
(二) 数据说明	1
(三) 实验口径说明	2
二、 前序实验方法整合	2
(一) SIMD: 将 Hash 阶段映射到 4 路 NEON	2
(二) 多线程: 优化 Guess 阶段的任务大小和队列维护	4
(三) 训练阶段: 利用统计可加性进行局部模型合并	5
(四) MPI: 多进程分工、流水线与训练分片	6
(五) GPU: 候选生成实验	8
(六) 小结	9
三、 MPI Hash 分发优化	9
(一) 每次分发的候选数量	9
(二) 字符串分发的耗时分析	11
(三) PT 模板和值范围分发	12
(四) 小结	13
四、 GPU 端生成 +MD5	13
(一) 设计目标与实现方案	13
(二) 主性能结果	14
(三) 分阶段开销分析	15
(四) GPU 任务阈值扫描	16
(五) 小结	17

五、 综合效果与正确性	17
(一) 阶段化加速效果	17
(二) 正确性与猜测质量	17
六、 总结	18
(一) 并行化思路	18
(二) 主要方法改进	18
(三) 结论	19

一、 问题描述与整合目标

(一) PCFG 口令猜测程序结构

本项目的口令猜测程序由三个主要阶段组成：首先从训练集统计 PCFG 模型；随后使用优先队列按概率顺序生成候选口令；最后对候选口令计算 MD5 哈希，或在正确性入口中与目标哈希集合进行匹配。三个阶段的数据依赖和并行特征并不相同，因此不能使用同一种并行方法统一处理。

1. **Train 阶段：**对训练集中的口令调用 `parse()`，统计 PT 模板频数和 segment value 频数。该阶段属于统计计数任务，数据具有可加性，可以拆分训练集并合并局部模型。
2. **Guess 阶段：**从优先队列取出概率最高的 PT，展开其最后一个 segment 的全部 value，并将新 PT 回插队列。最后一个 segment 的展开循环天然可并行，但全局优先队列维护具有串行依赖。
3. **Hash 阶段：**不同候选口令的 MD5 计算互不依赖，适合 SIMD、OpenMP 和 MPI 分发；但如果候选口令以 `string` 形式跨进程传输，通信和重建对象会带来额外开销。

前序实验分别从不同角度优化上述阶段，最后得到一条统一的技术路线：在节点内使用 SIMD 和 OpenMP 降低计算开销，在节点间使用 MPI 分发适合按较大任务切分的工作。

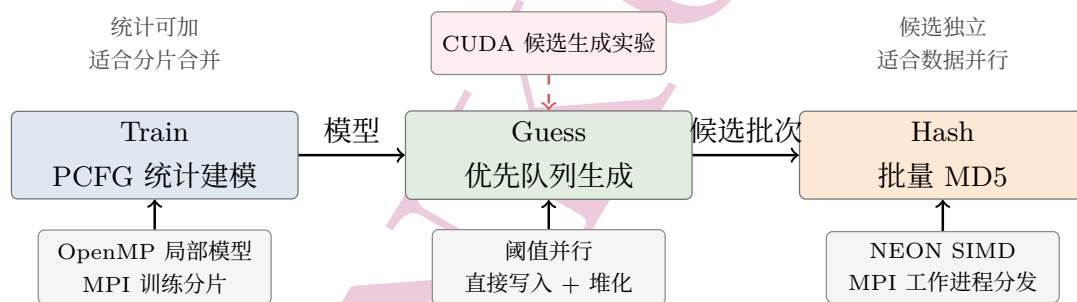


图 1: PCFG 口令猜测程序的阶段划分与对应并行技术

从程序流程看，Train 产生模型，Guess 根据模型和优先队列生成候选，Hash 对候选进行批量 MD5 计算并与目标集合匹配。并行化时，训练样本可以分别统计后合并，PT 的多个 value 可以分别展开，不同候选口令的哈希计算也可以同时进行。三个阶段的这些特点决定了后文所采用的并行方式。

(二) 数据说明

性能数据主要来自各实验报告中的 `main.cpp` 或 `mpi_main.cpp` 计时输出，正确性数据来自 `correctness_guess.cpp` 与 `mpi_correctness_guess.cpp` 的 Cracked 指标。所有实验使用课程集群提供的 `Rockyou-singleLined-full.txt` 数据集。

SIMD、多线程和 MPI 实验主要在课程服务器上完成；GPU 实验因学校集群无 CUDA 环境，在本地 WSL2 + RTX 4060 Laptop GPU 上完成。因此，GPU 实验数据只用于说明 CUDA 生成 + MD5 模块在同一台本地机器上相对 CPU/OpenMP 的效果，不与课程服务器上的数据做严格横向加速比比较。

(三) 实验口径说明

本文将各次实验放在同一条程序流程中进行比较，而不是只比较某一个独立函数。程序输入相同、候选生成上限相同，分别记录 Train、Guess 和 Hash 三个阶段的时间，并在 MPI 实验中进一步记录主进程发送、主进程等待、工作进程接收以及工作进程哈希等时间。

性能比较主要使用以下三个指标。首先，阶段时间直接反映某一部分程序的耗时变化；其次，整体加速比定义为

$$S = \frac{T_{\text{baseline}}}{T_{\text{parallel}}},$$

其中分母和分子使用相同的输入规模与计时范围。最后，正确性入口使用 Cracked 统计在给定候选数量内成功匹配的目标口令数量。对于不同入口或不同候选截断方式，本文不把数值差异解释为性能误差，而主要观察其数量级是否保持一致。

为了减少偶然波动，表格中的性能数据采用多次运行的平均值；代码优化前后保持编译选项和输入文件一致。GPU 数据来自不同硬件环境，因此只用于分析 CUDA 候选生成和 GPU 端生成 +MD5 的适用性，不参与服务器上 SIMD、OpenMP 和 MPI 的直接加速比排名。

二、 前序实验方法整合

(一) SIMD：将 Hash 阶段映射到 4 路 NEON

MD5 对单条消息内部存在严格的数据依赖，难以在单条口令内部并行。但不同口令之间没有数据依赖，因此 SIMD 实验采用“多条口令同步执行同一 MD5 步骤”的策略。实现上使用 ARM NEON 的 `uint32x4_t` 同时维护 4 条口令的 (A, B, C, D) 状态，并将 FF/GG/HH/II 四类轮函数改写为向量运算。

SIMD 部分围绕批量 MD5 计算进行了多方面改进。首先，程序使用 4 个 NEON lane 同时维护四条口令的 MD5 状态，将四条互不相关的口令组成一个计算批次；其次，将 MD5 的 FF、GG、HH、II 四类轮函数改写为向量运算，使四条口令能够同步执行相同的计算步骤；再次，将输入消息从按口令存放的 AoS 转换为便于向量装载的 SoA，以提高向量寄存器的数据装载效率。此外，批量哈希过程中使用栈上的临时缓冲区，减少循环中的动态内存分配，并配合编译器优化选项降低向量计算的额外开销。

其中，数据布局转换是 SIMD 实现中的关键工程问题。原始输入更接近 AoS，即每条口令独立保存 16 个 32 bit 字；而 SIMD 轮函数要求四条口令的同一个消息字进入同一个向量寄存器，因此需要进行 AoS 到 SoA 的转置。表 1 汇总了 SIMD 报告中完整程序 Hash time 的结果。

编译选项	串行 Hash time(s)	SIMD Hash time(s)	加速比
-O0	9.31	12.33	0.76×
-O1	3.00	1.11	2.70×
-O2	2.95	1.05	2.81×

表 1: SIMD 实验中完整程序 Hash time 对比 (10M 条口令)

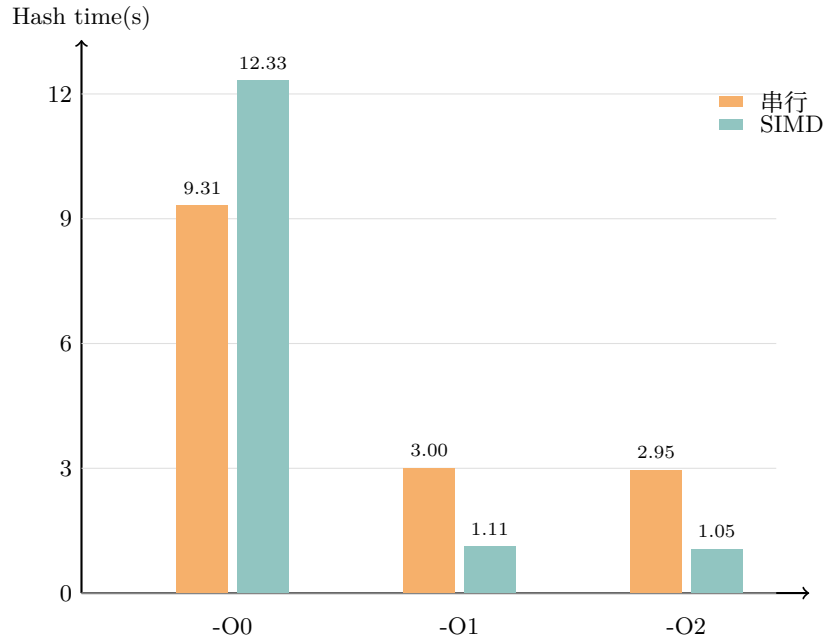


图 2: SIMD 实验中不同编译选项下的 Hash time 对比

从表 1 和图 2 可以看出, 在开启编译优化后, SIMD 版本的 Hash time 明显低于串行版本。-O2 下 Hash time 从 2.95s 降至 1.05s, 加速比为 2.81 倍; -O1 下也有 2.70 倍加速。只有 -O0 下 SIMD 版本更慢, 说明该实现依赖编译器对向量寄存器和临时变量的优化。总体来看, 批量 MD5 计算适合使用 SIMD 处理, 但实际效果需要配合合适的编译优化选项。

下面给出 SIMD 实现中数据转换和向量状态初始化的核心代码。四条指令先按照向量计算所需的形式装入寄存器, 后续 MD5 轮函数即可对四条指令同时执行。

Listing 1: NEON SIMD 哈希中的数据装载代码

```

1  uint32x4_t A = vdupq_n_u32(0x67452301);
2  uint32x4_t B = vdupq_n_u32(0xefcdab89);
3  uint32x4_t C = vdupq_n_u32(0x98badcfe);
4  uint32x4_t D = vdupq_n_u32(0x10325476);
5
6  uint32x4_t x[16];
7  for (int k = 0; k < 4; k++) {
8      int off = blk * 64 + k * 16;
9      uint32x4_t m0 = vld1q_u32((const uint32_t*)&padded[0][off]);
10     uint32x4_t m1 = vld1q_u32((const uint32_t*)&padded[1][off]);
11     uint32x4_t m2 = vld1q_u32((const uint32_t*)&padded[2][off]);
12     uint32x4_t m3 = vld1q_u32((const uint32_t*)&padded[3][off]);
13
14     uint32x4x2_t t0 = vzipq_u32(m0, m2);
15     uint32x4x2_t t1 = vzipq_u32(m1, m3);
16     uint32x4x2_t u0 = vzipq_u32(t0.val[0], t1.val[0]);
17     uint32x4x2_t u1 = vzipq_u32(t0.val[1], t1.val[1]);
18
19     x[k * 4 + 0] = u0.val[0];
20     x[k * 4 + 1] = u0.val[1];

```

```

21     x[k * 4 + 2] = u1.val[0];
22     x[k * 4 + 3] = u1.val[1];
23 }

```

这些优化之间并不是相互独立的，而是需要协同工作才能获得较好的性能。如果仅将 MD5 函数改写为向量运算，而保持原有 AoS 数据布局，则每一步计算前都需要频繁组织数据，向量装载会成为新的性能瓶颈；如果仅进行 AoS 到 SoA 转换，而不能连续处理大量指令，则数据转置本身的开销也难以摊销。实验中采用批量处理方式，使一次数据重排能够服务后续完整的 MD5 计算，同时利用栈缓冲区减少循环中的内存分配开销，从而进一步降低额外成本。

实验结果也验证了这一特点。-O0 下 SIMD 版本反而慢于串行版本，说明向量实现仍依赖编译器对寄存器分配、指令调度和临时变量优化；而在 -O1 和 -O2 下，这些额外开销得到有效控制，SIMD 并行计算的优势才充分体现出来。

(二) 多线程：优化 Guess 阶段的任务大小和队列维护

多线程实验主要针对 `PriorityQueue::Generate` 和 `PopNext`。对于一个已经确定前缀的 PT，最后一个 segment 的每个 value 都能独立生成一条候选指令，因此可以并行写入输出数组。但初版 OpenMP 直接使用局部 vector 再合并，反而因为小任务并行区开销、局部缓冲分配和二次拷贝导致 Guess time 上升。

最终版本采用三项优化：第一，对小 PT 串行处理，只在 `guess_count >= 200` 时启用 OpenMP；第二，提前 `resize` 输出数组，让线程按下标直接写入 `guesses[old_size+i]`，避免锁和合并；第三，将原始有序 vector 插入改为最大堆，使新 PT 插入和弹出从线性移动转为 $O(\log n)$ 堆操作。表 2 汇总了多线程报告中的优化路径。

下面是项目中 Guess 阶段的核心写入逻辑。程序先为结果数组预留空间，再根据任务大小选择串行循环或 OpenMP 循环，线程之间不需要合并临时数组。

Listing 2: Guess 阶段按任务大小选择并行方式

```

1  int guess_count = pt.max_indices[0];
2  size_t old_size = guesses.size();
3  guesses.resize(old_size + guess_count);
4
5  if (guess_count >= 200)
6  {
7      #pragma omp parallel for schedule(static)
8      for (int i = 0; i < guess_count; i++)
9      {
10         guesses[old_size + i] = a->ordered_values[i];
11     }
12 }
13 else
14 {
15     for (int i = 0; i < guess_count; i++)
16     {
17         guesses[old_size + i] = a->ordered_values[i];
18     }
19 }

```

版本	Guess time(s)	Hash time(s)	说明
串行基线 -02	0.573	2.948	原始生成 + 标量哈希
SIMD 哈希 -02	0.616	1.047	Hash 使用 SIMD, Guess 未并行
OpenMP 初版	1.115	1.035	无阈值 + 局部缓冲合并
OpenMP 阈值 + 直接写入	0.389	1.038	小任务串行, 避免临时数组合并
OpenMP 堆化	0.198	1.061	队列堆化, 主线实现
Pthread 4 线程	0.331	1.064	手写持久线程池
MultiQueue 版本	0.288	1.209	松弛并发优先队列

表 2: 多线程实验中 Guess 阶段优化路径 (main.cpp 平均值)

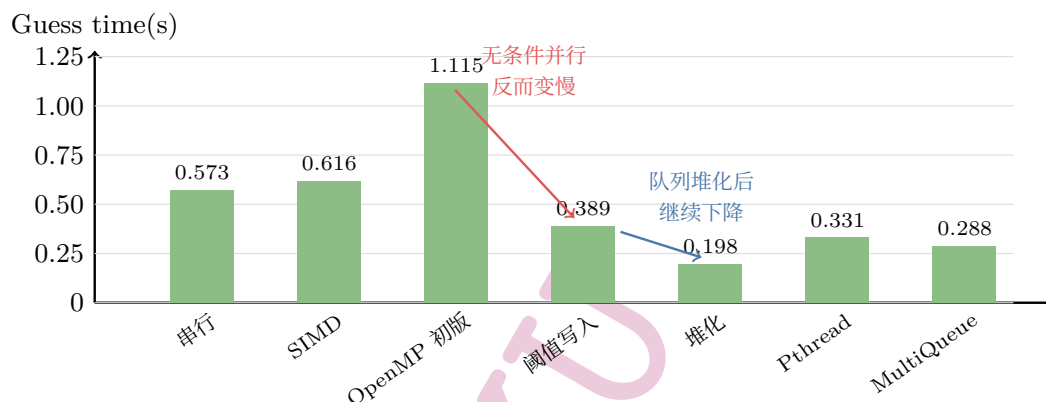


图 3: 多线程实验中 Guess 阶段优化路径可视化

从表 2 和图 3 可以看出, 初版 OpenMP 的 Guess time 为 1.115s, 比仅使用 SIMD 哈希的 0.616s 更慢, 说明对小任务无条件并行会引入较大的线程调度和合并开销。加入阈值判断和直接写入后, Guess time 降至 0.389s; 继续将优先队列改为堆结构后, Guess time 进一步降至 0.198s, 是该阶段效果最好的主线版本。Pthread 和 MultiQueue 分支也有一定效果, 但在当前数据规模下没有超过 OpenMP 堆化版本。

(三) 训练阶段：利用统计可加性进行局部模型合并

Train 阶段的优化思路是利用统计计数的可加性。PCFG 训练主要统计 PT 和 segment value 的频数, 因此可以将训练集按下标划分给多个线程, 分别构造局部模型, 最后再合并频数。

实现时, 每个线程只修改自己的局部模型, 避免训练过程中频繁访问共享模型。合并时不能直接按 value 的编号相加, 而要先根据 value 内容建立对应关系, 再累加频数, 最后重新排序模型。

采用这一方法后, Train time 由约 27.72s 降至约 20.55s, 继续使用 4 线程分片后降至约 11.95s; 对应的 Cracked=381892 保持不变, 说明训练并行化没有明显改变后续猜测结果。

项目中的局部模型训练代码如下。每个线程只写入自己的模型, 循环结束后再统一合并, 因此训练过程中不需要多个线程同时修改同一个模型。

Listing 3: Train 阶段的局部模型训练与合并

```

1 vector<model> partial_models(thread_count);
2

```



```

3  #pragma omp parallel num_threads(thread_count)
4  {
5      int tid = omp_get_thread_num();
6      size_t begin = passwords.size() * tid / thread_count;
7      size_t end = passwords.size() * (tid + 1) / thread_count;
8
9      for (size_t i = begin; i < end; i += 1)
10     {
11         partial_models[tid].parse(passwords[i]);
12     }
13 }
14
15 for (model &partial : partial_models)
16 {
17     merge_model(*this, partial);
18 }

```

(四) MPI：多进程分工、流水线与训练分片

MPI 实验把前序 SIMD 和 OpenMP 优化放入分布式框架中。基础结构采用主进程-工作进程模式：主进程负责训练、维护优先队列和生成候选口令，工作进程接收候选批次并执行 OpenMP + SIMD 哈希。该设计保留了全局候选生成顺序，避免多个进程重复生成同一搜索空间。

由于候选口令是变长字符串，MPI 通信采用三段式协议：先发送数量，再发送长度数组，最后发送拼接后的字符缓冲区。工作进程重建 `vector<string>` 后进行本地哈希。这个协议稳定但有额外打包、传输和重建开销，因此 MPI 主要适合批量较大的 Hash 分发。后续版本加入进程内 OpenMP 哈希分片、生成-哈希流水线以及 Train 阶段 MPI 分片。

主进程保留优先队列，因此候选仍按原程序的概率顺序产生；工作进程只处理已经分配的候选批次。发送一个批次时，主进程先记录每条口令的长度，再将所有字符复制到连续缓冲区；工作进程接收后根据长度逐条恢复字符串，再调用本地的 OpenMP+SIMD 哈希函数。该方法能够正确传输变长口令，但打包和恢复字符串也会带来额外耗时。

下面给出原字符串分发方法中工作进程接收数据的核心代码。长度数组只保存每条口令的边界，字符缓冲区保存实际内容；接收端通过偏移量依次恢复每个字符串。

Listing 4: MPI 工作进程接收变长候选口令

```

1  vector<string> receive_password_batch(int count)
2  {
3      vector<int> lengths(count);
4      MPI_Recv(lengths.data(), count, MPI_INT, 0, TAG_WORK,
5              MPI_COMM_WORLD, MPI_STATUS_IGNORE);
6
7      size_t total_chars = 0;
8      for (int len : lengths)
9      {
10         if (len < 0)
11         {
12             MPI_Abort(MPI_COMM_WORLD, 3);
13         }

```

```

14     total_chars += static_cast<size_t>(len);
15 }
16
17 vector<char> buffer(total_chars);
18 if (!buffer.empty())
19 {
20     MPI_Recv(buffer.data(), static_cast<int>(buffer.size()),
21             MPI_CHAR, 0, TAG_WORK, MPI_COMM_WORLD,
22             MPI_STATUS_IGNORE);
23 }
24
25 vector<string> passwords;
26 passwords.reserve(count);
27 size_t offset = 0;
28 for (int len : lengths)
29 {
30     if (len == 0)
31     {
32         passwords.emplace_back();
33     }
34     else
35     {
36         passwords.emplace_back(buffer.data() + offset,
37                                static_cast<size_t>(len));
38     }
39     offset += static_cast<size_t>(len);
40 }
41 return passwords;
42 }

```

从并行结构上看，主进程-工作进程模式的优点是保留了全局候选生成顺序，缺点是主进程需要承担队列维护、候选生成和数据发送三项工作。当本地 SIMD 哈希已经很快时，工作进程的计算时间可能小于主进程的打包和通信时间，增加工作进程数量并不会自动带来加速。这也是本文后续继续分析 MPI Hash 分发方式的原因。

版本	Guess(s)	Hash(s)	Train(s)	总时间 (s)	相对串行加速
最基本串行算法	0.569061	2.856280	27.112800	30.538141	1.00×
OpenMP 训练并行化 (4 线程)	0.266669	1.054560	11.904100	13.225329	2.31×
MPI 单进程对照 NP=1,OMP=8	0.137739	0.129025	10.998333	11.265097	2.71×
MPI 训练分片 + 流水线 NP=4,OMP=8	0.142894	0.217669	8.271353	8.631917	3.54×

表 3: MPI 报告中的整体性能对比

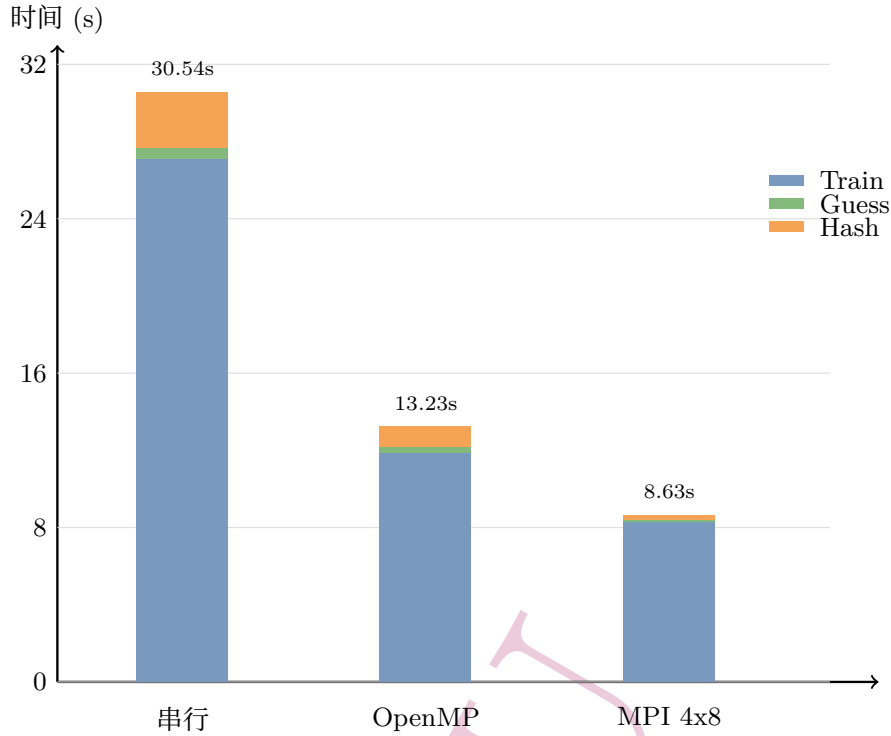


图 4: 串行、OpenMP 与 MPI 最终版本的阶段耗时堆叠对比

从表 3 和图 4 可以看出，MPI 最终版本的平均总时间为 8.631917s，相比最基本串行算法 30.538141s 有明显下降。与 NP=1,OMP=8 单进程对照相比，NP=4,OMP=8 的 Train time 从 10.998333s 降至 8.271353s，是总时间继续下降的主要来源。但该配置下 Hash time 反而高于单进程对照，说明当本地 SIMD+OpenMP 哈希已经较快时，跨进程传输字符串、打包和等待工作进程会带来额外开销。这一结果也引出了后文对 MPI Hash 通信成本的进一步分析。

(五) GPU：候选生成实验

GPU 实验将 `PriorityQueue::Generate` 中最后一个 segment 的 value 展开迁移到 CUDA。Host 侧把 `vector<string>` 中的 values 打包为连续字符缓冲区、offset 和 length；Device 侧一线程生成一条 `prefix + value[i]`；拷回 Host 后再恢复为 `std::string`。该前序实验保留 CPU/OpenMP 回退路径，当时使用 `GPU_GENERATE_THRESHOLD=4096`，小任务仍走 CPU。

程序还实现了 CPU/GPU 重叠：GPU 后台生成候选口令时，CPU 主线程继续执行 `NewPTs()`、概率计算和堆维护，最后等待 GPU 结果。表 4 汇总了 GPU 报告中的本地平台平均结果。

版本	Guess 平均 (s)	Hash 平均 (s)	Train 平均 (s)	总时间估计 (s)
CPU/OpenMP 基线	1.07888	2.21443	10.02303	13.31634
同步 CUDA	2.61463	2.21730	10.30260	15.13454
CUDA 重叠	2.56730	2.21340	10.34200	15.12270

表 4: GPU 报告中的本地平台主性能实验平均结果

从表 4 可以看出，同步 CUDA 版本的 Guess 平均时间为 2.61463s，高于 CPU/OpenMP 基线的 1.07888s；加入 CPU/GPU 重叠后，Guess time 降至 2.56730s，但仍未超过 CPU/OpenMP。

造成这一结果的主要原因是候选口令生成包含较多字符串打包、传输和恢复操作，而这些操作的开销抵消了 GPU 并行展开 value 的收益。因此，本实验表明低计算量但包含大量字符串处理的生成任务，单独迁移到 GPU 未必能够获得加速。

(六) 小结

在最终整合版本中，各种并行方法分别作用于不同阶段：MPI 负责进程之间的任务分配，OpenMP 负责进程内部的训练、候选展开和哈希，SIMD 负责批量计算多条口令的 MD5，CUDA 用于分析候选生成在 GPU 上的适用性。MPI 工作进程收到任务后直接使用本地的 OpenMP+SIMD 路径，不重复维护优先队列；主进程继续负责全局候选顺序和任务分发。

使用位置	主要工具	在本项目中的作用
进程之间	MPI	分发训练数据或候选任务，汇总局部结果
进程内部	OpenMP	并行训练、value 展开和本地哈希
向量计算	ARM NEON SIMD	同时计算多条候选口令的 MD5
CPU 与 GPU 之间	CUDA	尝试并行生成候选字符串

表 5: 本项目中不同并行方法的使用位置

从结果看，Train 阶段的训练分片对完整程序总时间影响最大，SIMD Hash 和 OpenMP Guess 优化分别改善了对应阶段。GPU 实验显示，单独把字符串生成放到 GPU 会受到打包、传输和重建字符串的限制。PT 模板和值范围分发则减少了 MPI Hash 中重复传输候选字符串的开销。不同方法在最终程序中分别服务于不同阶段。

三、 MPI Hash 分发优化

前文的 MPI 融合版本已经将 SIMD、OpenMP、主从式分发、生成-哈希流水线和训练集分片结合到同一入口中。该版本的总时间低于单进程对照，主要收益来自 Train 阶段分片；但 Hash time 仍高于 MPI_NP=1, OMP_NUM_THREADS=8 的本地哈希路径。表 3 展示的是前序 MPI 融合版本的完整程序性能，本节针对 Hash 阶段的任务分发方式进行改进，重点分析每次发送多少候选口令、字符串通信的耗时，以及最终的 PT 模板和值范围分发方案。

(一) 每次分发的候选数量

原 MPI 版本使用固定的 HASH_BATCH_THRESHOLD=1000000。这类固定阈值不能反映不同工作进程数量和通信开销下的任务大小需求，因此本阶段将阈值改为运行时策略：未显式设置环境变量时，单进程保持 1000000；多进程运行时，按照“每个工作进程约 250000 条候选口令”的原则估计总批量，并限制在 250000 到 2000000 之间。例如 MPI_NP=4 时，1 个主进程负责调度，3 个工作进程负责哈希，默认批量约为 $250000 \times 3 = 750000$ 。实验时也可以通过 HASH_BATCH_SIZE 覆盖该值。

早期测试中，达到候选数量上限时，缓冲区中尚未发送的最后一批候选没有被及时处理，导致统计出的 Hash time 不正确。程序随后在结束前统一处理剩余候选，表 6 给出了修正后的实验结果。

当前程序根据工作进程数量设置默认的每次分发数量，同时保留环境变量覆盖入口。下面的代码是该策略的实际实现。

Listing 5: 根据工作进程数量设置分发数量

```

1 if (world_size <= 1)
2 {
3     return DEFAULT_HASH_BATCH_THRESHOLD;
4 }
5
6 int worker_count = world_size - 1;
7 int adaptive_threshold = HASH_BATCH_PER_WORKER * worker_count;
8 return max(MIN_ADAPTIVE_HASH_BATCH_THRESHOLD,
9           min(MAX_ADAPTIVE_HASH_BATCH_THRESHOLD, adaptive_threshold));

```

配置	Guess 平均 (s)	Hash 平均 (s)	Train 平均 (s)	总时间估计 (s)
NP=1,OMP=8	0.134	0.128	10.613	10.875
NP=4,OMP=8,250k	0.181	1.020	8.184	9.386
NP=4,OMP=8,500k	0.162	1.018	8.401	9.581
NP=4,OMP=8,1M	0.189	1.025	8.385	9.599
NP=4,OMP=8,2M	0.238	1.019	8.407	9.664
NP=4,OMP=8,5M	0.237	1.093	8.366	9.696
NP=4,OMP=8,10M	0.355	1.211	8.271	9.837

表 6: 不同候选分发数量下的 Hash time



图 5: 不同候选分发数量下的 Hash time

从表 6 和图 5 可以看出, 250k 到 2M 之间的 Hash time 差异并不显著, 说明在当前程序规模下, 单纯调整每次分发的候选数量不能从根本上改变 MPI Hash 阶段的性能。5M 和 10M 的 Hash time 上升更明显, 原因是单批候选过大时, 主进程需要先积累、打包并连续发送更多变长字符串, 同时批次数减少, 生成与哈希之间可重叠的机会也随之减少。该版本每次都会向全部工

作进程分配候选，因此性能下降不是由工作进程未获得任务造成的。另一方面，较小的分发数量虽然通信次数更多，但每次数据量较轻，因此并未出现明显劣化。该结果提示，每次分发多少候选口令会影响性能，但更核心的问题需要通过更细的分阶段计时定位。

(二) 字符串分发的耗时分析

为了明确 Hash time 的来源，本文在原字符串分发方法上增加了分阶段计时。主进程记录优先队列初始化、PopNext() 与候选生成、批量打包发送、等待工作进程返回；工作进程记录等待任务、接收并重建候选字符串、本地哈希和返回完成消息。表 7 给出代表性结果。

表 7 的主要作用是观察 Hash 阶段由哪些部分构成。不同配置同时改变了进程数、每进程线程数和进程分布位置，因此表中时间的长短还会受到通信路径的影响。例如，NP=4,OMP=8 时四个进程分布在不同节点，而 NP=4,OMP=2 时多个进程可能位于同一节点；两种情况下的 MPI 通信路径不同，发送和接收时间也会不同。因此，表中数据用于说明耗时来源，不用于单独比较线程数的优劣。

配置	Hash(s)	主进程发送 (s)	主进程等待 (s)	工作进程接收均值 (s)	工作进程哈希均值 (s)	主进程分发次数
NP=4,OMP=8,1M	1.02774	0.84553	0.18221	0.59719	0.04543	10
NP=2,OMP=4,1M	0.46458	0.19484	0.26975	0.30366	0.26543	10
NP=4,OMP=2,1M	0.22616	0.19232	0.03384	0.11856	0.17980	10
NP=4,OMP=8,500k	1.06457	0.93620	0.12836	0.52916	0.04590	19
NP=4,OMP=8,5M	1.08739	1.03441	0.05299	0.44663	0.04543	2

表 7: 原字符串分发方法的 Hash 阶段耗时分析

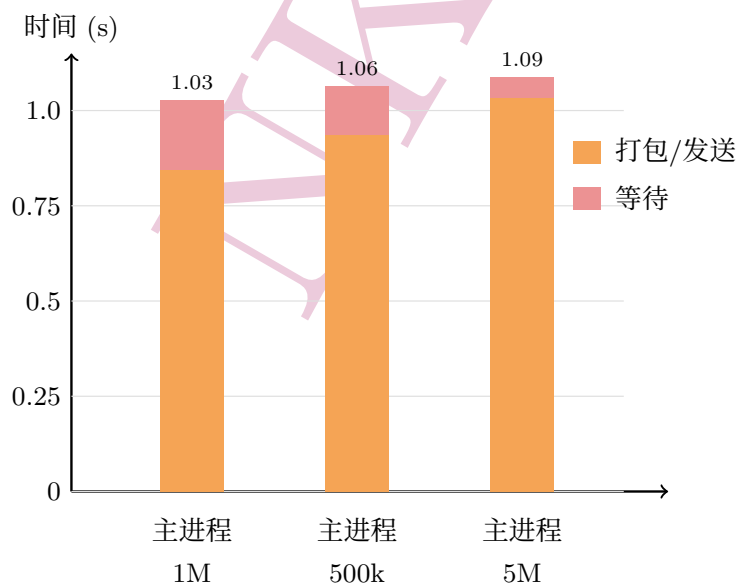


图 6: 原方法中主进程 Hash 阶段耗时拆分

分阶段计时结果支持前文对 MPI Hash 分发的判断：在 NP=4,OMP=8,1M 下，工作进程平均纯哈希时间只有 0.04543s，而主进程的打包发送时间为 0.84553s，工作进程平均接收和重建字符串时间为 0.59719s。因此，当前 Hash time 主要不是 MD5 计算本身，而是变长字符串的打包、跨进程传输和 `vector<string>` 重建。该结论也解释了为什么 NP=1,OMP=8 的本地 Hash time 只有约 0.128s：单进程路径避免了上述 MPI 字符串通信。

(三) PT 模板和值范围分发

基于上述计时结果，最终方案对候选口令的分发方式进行了修改。原方法先生成完整候选口令，再将大量变长字符串发送给工作进程；新方法保留主进程维护全局优先队列的设计，但只发送 PT 模板和最后一个 segment 的 value 起止下标。工作进程收到任务后，根据本地模型生成对应范围的候选口令并完成哈希。

这样可以减少字符串打包、传输和重建的开销。由于不同 PT 展开的候选数量可能差别较大，程序进一步把一个 PT 的值范围切成多个区间，让不同工作进程分别处理，减少任务分配不均的问题。前期也比较过按完整 PT 分配任务的方式，最终采用值范围作为分配单位。

最终任务中保存的是 PT 以及 value 的起止位置，发送时只需打包这些信息。对应代码如下。

Listing 6: PT 模板和值范围任务的构造

```

1 struct PTSliceTask
2 {
3     PT pt;
4     int value_begin = 0;
5     int value_end = 0;
6 };
7
8 void append_pt_slice_task(vector<char> &buffer,
9                          const PTSliceTask &task)
10 {
11     append_pt(buffer, task.pt);
12     append_int32(buffer, task.value_begin);
13     append_int32(buffer, task.value_end);
14 }

```

方法	Hash(s)	主进程发送 (s)	主进程等待 (s)	工作进程处理量分布
字符串分发 500k	1.06457	0.93620	0.12836	—
PT 模板和值范围分发	0.08402	0.00284	0.08119	3166694/3499986/3333320

表 8: PT 模板和值范围分发后的 Hash 阶段结果

表 8 对比了原字符串分发和最终方案。可以看到，PT 模板和值范围分发后，Hash time 从 1.06457s 降至 0.08402s，约为原来的 7.9%；主进程发送时间从 0.93620s 降至 0.00284s，说明通信开销明显下降。三个工作进程处理的候选数量分别为 3166694、3499986、3333320，分布较为接近，说明把一个 PT 的值范围分开处理也缓解了任务分配不均的问题。进一步统计工作进程内部时间后，测得工作进程平均工作时间为 0.0849853s，与主进程等待时间 0.08119s 基本接近，说明 Hash 阶段的计时已经能够较好反映工作进程的实际计算过程。

PT 模板和值范围分发要求工作进程提前准备好模型。工作进程接收模型后还要进行反序列化和排序；如果直接开始 Hash 计时，这段准备时间会混入主进程的等待时间。为统一计时范围，程序让所有进程完成模型准备后再开始正式的 Hash 分发。因此，表格和图中的 Hash time 只统计候选任务分发之后的通信、等待和哈希时间。

模型广播、反序列化和排序只在开始阶段进行一次，而完整候选字符串的打包、传输和重建会在后续批次中反复发生。对于单轮 10M 候选实验，模型准备时间仍然属于完整程序总时间；PT 模板和值范围分发的优势主要体现在减少后续批次的重复通信。

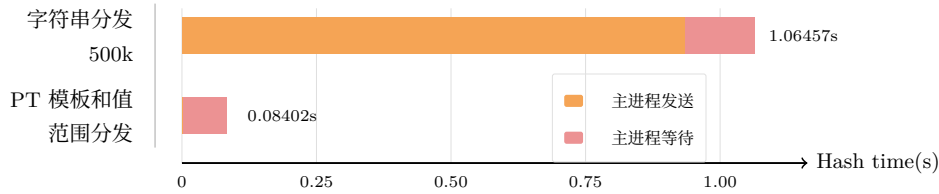


图 7: 原字符串分发与 PT 模板和值范围分发的 Hash 阶段开销对比

与原字符串分发 500k 相比, PT 模板和值范围分发的 Hash time 从 1.06457s 降至 0.084021s, 约为原来的 7.9%。其中主进程发送时间由 0.93620s 降至 0.002835s, 说明发送 PT 模板和值范围, 显著减少了反复传输完整候选字符串的开销。在完成模型准备后, 新的分发方式能够明显降低 Hash 阶段的通信和等待成本; 模型准备本身仍应计入完整程序的总时间。

(四) 小结

综上, 本阶段最终采用 PT 模板和值范围分发来代替完整字符串分发。该方法减少了主进程反复打包和发送候选字符串的开销, 也让工作进程能够更均衡地处理候选口令。实验中 Hash time 从 1.06457s 降至 0.084021s, 主进程发送时间从 0.93620s 降至 0.002835s, 说明该方案对 Hash 分发阶段的通信开销有明显优化效果。

四、 GPU 端生成 +MD5

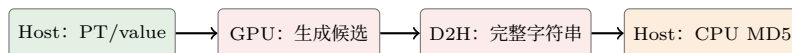
前序 GPU 实验只在 Device 端生成候选字符串, 随后仍需把完整字符串传回 Host、恢复为 `std::string`, 并在 CPU 上计算 MD5。该方案的主要开销来自字符串搬运和对象重建, 而不是 GPU 计算。因此, 本节新增 GPU 端生成 +MD5 方案: GPU 线程生成候选后立即计算其 MD5, 只向 CPU 回传固定长度的 128-bit 摘要。

Host 仍负责维护 PCFG 优先队列; 它向 GPU 传入公共前缀和最后一个 segment 的连续 value 缓冲区。每个 GPU 线程处理一个 value, 完成逻辑拼接、MD5 填充和单块 MD5 压缩。候选长度超过 55 字节时, 当前实现自动回退到原有 CPU/OpenMP 路径, 以保证结果正确。

(一) 设计目标与实现方案

图 8 对比了旧 GPU 方案和改进后 GPU 端生成 +MD5 方案。旧方案中, GPU 仅负责生成候选字符串, 完整字符串仍需传回 Host, 并恢复为 `std::string` 后由 CPU 计算 MD5; 改进方案则在 GPU 上完成候选生成和 MD5 计算, 只向 Host 回传固定长度的 128-bit 摘要, 从而避免大量字符串回传和对象重建。

旧方案：回传完整字符串



改进方案：只回传摘要



图 8: 旧 GPU 路径与 GPU 端生成 +MD5 路径的数据流对比

核心 kernel 的每个线程只处理一条候选。线程根据 prefix 和当前 value 在本地构造一个 64 字节 MD5 数据块, 完成填充后执行单块 MD5 压缩; 长度超过 55 字节的候选不进入该 kernel,

而是交由 CPU/OpenMP 路径处理。

Listing 7: GPU 端生成 +MD5 kernel 的核心逻辑

```

1 __global__ void generate_hash_kernel(...) {
2     int idx = blockIdx.x * blockDim.x + threadIdx.x;
3     if (idx >= count) return;
4     int len = prefix_len + lengths[idx];
5     if (len > 55) return;          // 交由 CPU 路径处理
6     unsigned char block[64] = {};
7     copy_prefix_and_value(block, prefix, values, offsets[idx], lengths[idx]);
8     build_md5_padding(block, len);
9     md5_compress_one_block(block, out + 4 * idx);
10 }

```

Host 侧使用可复用的 FusedWorkspace 保存 prefix、value、offset、length 和摘要缓冲区；只有容量不足时才扩容，从而避免每个 PT 反复调用 cudaMalloc 和 cudaFree。

(二) 主性能结果

CPU/OpenMP、旧 GPU 生成和新增 GPU 端生成 +MD5 路径均在同一台本地 RTX 4060 Laptop GPU 环境下运行三次；GPU 端生成 +MD5 在本表中固定使用 4K 阈值，以便与此前的 CPU 和旧 GPU 对照结果对应。表中总时间为 Guess time 与 Hash time 之和，不含 Train time。16K 阈值的五次重复测量在后文单独给出。三条路径的主性能结果如表 9 和图 9 所示。

路径	Guess 平均 (s)	Hash 平均 (s)	总时间平均 (s)	说明
CPU/OpenMP	0.261823	1.182380	1.444203	本地 CPU 基线
旧 GPU 生成	2.128203	1.169997	3.298200	GPU 只生成字符串
GPU 端生成 +MD5 (4K)	1.322170	0.099175	1.421345	只回传摘要

表 9: GPU 端生成 +MD5 的主性能结果 (10M 候选, 三次平均)

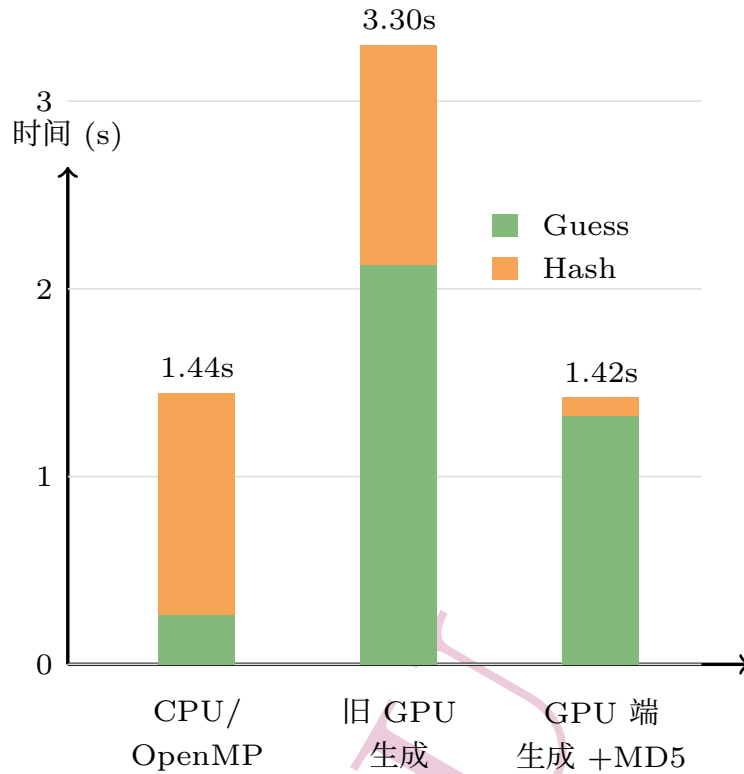


图 9: 三条路径的 Guess/Hash 时间对比

旧 GPU 生成路径没有减少 CPU 端 MD5, 因此 Guess time 上升到 2.13s, 总时间达到 3.30s。GPU 端生成 +MD5 路径的总时间为 1.421345s, 略低于 CPU/OpenMP 的 1.444203s。

需要注意 GPU 端生成 +MD5 路径的阶段计时口径发生了变化。CPU/OpenMP 路径在批量刷新时执行 MD5Hash_SIMD(), 所以 MD5 计入 Hash time; GPU 路径在 PopNext() 内同步完成 value 打包、H2D、GPU kernel 内的候选生成和 MD5、D2H 摘要回传、摘要整理以及 PT 队列推进, 这些都计入 Guess time。性能入口不进行摘要匹配, GPU 摘要只被保存后清理; Hash time 仅包含 CPU 小 PT、超长候选和 CUDA 失败回退候选的 CPU MD5, 以及少量批量清理开销。因此, 表中的 Guess time 和 Hash time 不能在 CPU 与 GPU 两条路径之间逐项比较; 应以总时间判断端到端性能, 并用后文的 GPU 分阶段计时解释开销来源。

(三) 分阶段开销分析

为定位 GPU 路径中 Guess time 增大的来源, 程序进一步记录了打包、H2D、kernel 和 D2H 的累计时间。其中, H2D (Host to Device) 指 CPU 内存到 GPU 显存的传输, D2H (Device to Host) 指 GPU 显存到 CPU 内存的传输。

项目	Pack(s)	H2D(s)	Kernel(s)	D2H(s)
三次平均	0.069329	0.056952	0.012856	0.037785

表 10: GPU 端生成 +MD5 的分阶段计时 (阈值 4096, 10M 候选)



图 10: GPU 路径的分阶段累计耗时；右侧图例给出各阶段时间

从表 10 和图 10 可以看出, kernel 仅为 0.012856s, 不是主要瓶颈; 打包和两次传输合计约 0.164s。其余时间主要来自 PT 队列推进、小 PT 的 CPU 回退、频繁 kernel launch 和批处理管理。

(四) GPU 任务阈值扫描

分阶段计时表明, 当前主要开销并不在 kernel 本身, 而在小任务的打包、传输和提交。因此, 本节进一步扫描 GPU 任务阈值, 考察何种 PT 规模下将任务交给 GPU 更合适。

GPU_GENERATE_THRESHOLD 表示一个 PT 展开出的候选数量达到多少时, 程序才将该 PT 交给 GPU 端生成并计算 MD5; 低于该值的 PT 保留在 CPU/OpenMP 路径。阈值较低时, 更多 PT 会进入 GPU, 但会增加数据打包、传输和 kernel launch 次数; 阈值较高时, GPU 调用次数下降, 却会使更多候选回退到 CPU。为避免单次运行受到偶然因素影响, 本节在相同数据集和 10M 候选上限下, 对四个阈值各重复运行 5 次, 结果如表 11 所示。

阈值	第 1 次 (s)	第 2 次 (s)	第 3 次 (s)	第 4 次 (s)	第 5 次 (s)	中位数 (s)
4096	2.766172	2.355506	1.288626	1.302990	1.419034	1.419034
16384	1.819419	1.564792	1.350015	1.328978	1.384126	1.384126
65536	2.674604	1.618311	1.735538	1.648276	8.854878	1.735538
262144	4.852270	1.408197	1.448423	1.401488	1.419276	1.419276

表 11: GPU 端生成 +MD5 的阈值重复测量结果 (10M 候选, 每组 5 次)

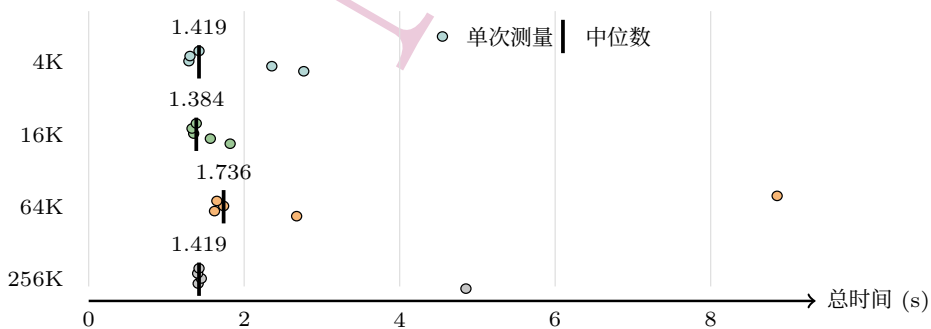


图 11: 不同 GPU 任务阈值下的原始测量结果与中位数 (每组 5 次)

图 11 展示了每组的 5 次原始测量结果, 黑色粗线表示中位数。16K 的中位数最低, 为 1.384126s; 4K 和 256K 的中位数分别为 1.419034s 和 1.419276s, 与 16K 的差距均约为 0.035s。由于每组仅测量 5 次, 且部分运行存在较明显波动, 该差距尚不足以说明 16K 具有稳定且显著的性能优势。因此, 本实验只将 16K 视为当前实现下取得最低中位数的候选默认阈值。

4K 的前两次测量分别为 2.766172s 和 2.355506s, 明显慢于其余三次; 这是可能受到 CUDA 初始化、缓存预热或系统负载的影响。64K 的中位数升至 1.735538s, 且出现一次 8.854878s 的

异常慢次；除该异常值外，64K 的多数结果也整体慢于 16K，说明阈值过高后更多 PT 回退到 CPU 路径，可能削弱了 GPU 的收益。256K 下没有 PT 触发 GPU 路径，除一次 4.852270s 的异常测量外，其余结果集中在 1.40–1.45s，接近纯 CPU/OpenMP 路径。

(五) 小结

GPU 端生成 +MD5 将短候选的 MD5 移入 GPU kernel，并取消明文字符串回传。4K 对照中，端到端总时间由 1.444203s 降至 1.421345s；结合阈值扫描，16K 取得了最低的总时间中位数。当前主要性能瓶颈已从 MD5 计算转移到 Host 侧打包、数据传输、任务提交和 PT 队列管理。

五、 综合效果与正确性

(一) 阶段化加速效果

将前序实验合并后，可以看到不同并行方法分别作用于不同阶段。SIMD 显著压缩 Hash time；OpenMP 阈值并行和堆化显著压缩 Guess time；Train 的 OpenMP/MPI 分片决定完整运行的总时间上限；GPU 端生成 +MD5 通过减少明文字符串回传降低了端到端总时间。表 12 按阶段总结已有结果。

阶段	有效方法	已记录效果
Hash	ARM NEON 4 路 SIMD, 批量 MD5	-02 下 Hash time 2.95s → 1.05s, 2.81×
Guess	OpenMP 阈值并行、直接写入、堆化优先队列	仅 SIMD 哈希时 Guess time 0.616s → 0.198s, 约 3.11×
Train	OpenMP 局部模型、MPI 训练集分片与模型合并	MPI NP=4, OMP=8 中 Train time 8.271353s, 总时间 8.631917s
CUDA 候选生成	一线程生成一条候选口令、CPU/GPU 重叠	本地 Guess time 2.61463s → 2.56730s, 但仍慢于 CPU/OpenMP
GPU 端生成 +MD5 (4K 对照)	GPU 端完成短候选拼接和单块 MD5, 只回传摘要	Hash+Guess 时间 1.444203s → 1.421345s

表 12: 前序实验按阶段整合后的效果总结

(二) 正确性与猜测质量

本项目中有两类正确性：MD5 哈希本身的正确性和 PCFG 候选集合/猜测质量的相对正确性。SIMD 实验使用标准 MD5 测试向量验证哈希输出；Guess/MPI/GPU 相关实验使用 Cracked 指标验证候选生成和分发没有明显偏离。

版本或实验	Cracked	说明
OpenMP 堆化版本	382853	多线程报告主线版本
MultiQueue 平均值	381787	5 次运行平均, 松弛队列版本
MultiQueue 最小值	380972	5 次运行最低值
串行 / SIMD / OpenMP 训练修复后结果	381892	训练统计修复后的验证结果
MPI 正确性验证	381892	<code>mpi_correctness_guess.cpp</code>
期末 PT 模板和值范围正确性验证	382497	候选数量为 10M 的验证结果
CUDA 重叠正确性验证	381892	本地 CUDA 生成与 CPU/OpenMP 一致
GPU 端生成 +MD5 验证	382498	10M 候选下 CPU、旧 CUDA 与新增路径一致

表 13: 前序实验中的 Cracked 指标汇总

从表 13 可以看出, 各版本的 Cracked 结果都保持在 38 万左右, 数量级基本一致。其中期末 PT 模板和值范围正确性验证得到 Cracked=382497, 与前序主线结果接近, 说明新的任务分发方式没有明显破坏候选生成和哈希匹配的有效性。

六、 总结

(一) 并行化思路

PCFG 口令猜测程序的 Train、Guess 和 Hash 三个阶段具有不同的数据依赖, 因此并行化时需要采用不同的任务划分方式。

Hash 阶段中, 不同候选口令的 MD5 计算相互独立, 适合使用 SIMD 进行批量计算, 也可以通过 MPI 将候选任务分发给多个进程。Guess 阶段只有最后一个 segment 的 value 展开能够直接并行, 而优先队列仍需要按概率顺序维护, 因此优化重点不仅是增加线程, 还包括控制并行任务大小和降低队列维护成本。Train 阶段主要进行频率统计, 局部统计结果可以合并, 适合让不同线程或进程分别训练局部模型, 最后统一汇总。

(二) 主要方法改进

在各阶段的基础并行实现上, 本项目进一步进行了以下改进:

1. 在 SIMD Hash 中完成 AoS 到 SoA 的数据布局转换, 使多条候选口令能够按照 NEON 向量指令需要的形式同时计算。
2. 在 Guess 阶段加入任务规模阈值, 小规模展开继续串行执行; 同时让线程直接写入预先分配的结果数组, 避免局部缓冲区合并和额外拷贝。
3. 将原有优先队列改为堆结构, 使 PT 的插入和取出复杂度降低为 $O(\log n)$; 此外还尝试使用 MultiQueue 提高 PT 层的并行度, 并通过 Cracked 指标检查顺序放松对结果的影响。
4. 在 Train 阶段使用局部模型进行无锁统计, 最后合并频数, 并处理不同局部模型之间的 value id 映射问题。

5. 在 MPI 实验中加入生成-哈希流水线和分阶段计时，定位出完整字符串的打包、通信和重建是主要开销。最终改为发送 PT 模板和值范围，使工作进程在本地生成候选并完成哈希，同时改善不同工作进程之间的负载分配。
6. 在 GPU 实验中保留 CPU/OpenMP 回退路径，并尝试让 GPU 候选生成与 CPU 上的概率计算和队列维护重叠执行。在期末新增实验中，进一步把候选生成和单块 MD5 放入同一个 CUDA kernel，减少明文候选字符串的 D2H 回传和 Host 端 `std::string` 重建。

部分并行版本没有获得稳定正加速。例如，小任务直接使用 OpenMP 会增加调度开销，旧 CUDA 候选生成会受到字符串打包和数据传输的限制，MPI 完整字符串分发也会产生较高的通信成本。这些结果说明，并行性能取决于计算收益能否覆盖线程调度、同步、通信和数据搬运等额外开销，而不是简单增加线程数、进程数或计算设备。

(三) 结论

本学期的并行程序设计课程中，我围绕 PCFG 口令猜测程序完成了 SIMD、多线程、MPI 和 GPU 编程实验，并在期末阶段对各部分内容进行了整合。实验中，SIMD 降低了批量 MD5 计算的时间；OpenMP 阈值并行、直接写入和优先队列堆化改善了 Guess 阶段的性能；局部模型训练和 MPI 分片缩短了 Train 阶段的运行时间；PT 模板和值范围分发减少了 MPI Hash 阶段的通信开销；GPU 端生成 +MD5 实验通过减少明文字符串回传，进一步降低了端到端总时间。

通过这些实验，我对并行程序有了更加完整的理解。并行优化并不只是增加线程数或进程数，还需要考虑任务之间的数据依赖、每次处理的任务大小、数据布局、同步开销和通信成本。某个阶段被加速后，程序的主要瓶颈也可能转移到其他阶段，因此需要结合实际计时结果不断分析和调整。

在实现和优化的过程中，我也更加熟悉了 SIMD、多线程、MPI 和 GPU 编程的基本方法，并进一步认识到数据结构、内存布局和任务划分对并行性能的重要影响。相比只关注最终的加速比，分析性能变化的原因，以及理解不同并行方法的适用场景，同样是本次课程的重要收获。

总的来说，我在这门课程中学习到了很多并行程序设计方面的知识，也积累了较为完整的并行程序分析和优化经验。感谢老师的授课，也感谢助教对我提出的问题给予及时解答。最后，祝老师和助教工作顺利、万事如意。

附录

AI 使用说明

本次作业使用 Codex 辅助完成方案讨论、代码生成、实验数据整理。相关对话记录保存在 AI 对话记录.txt。